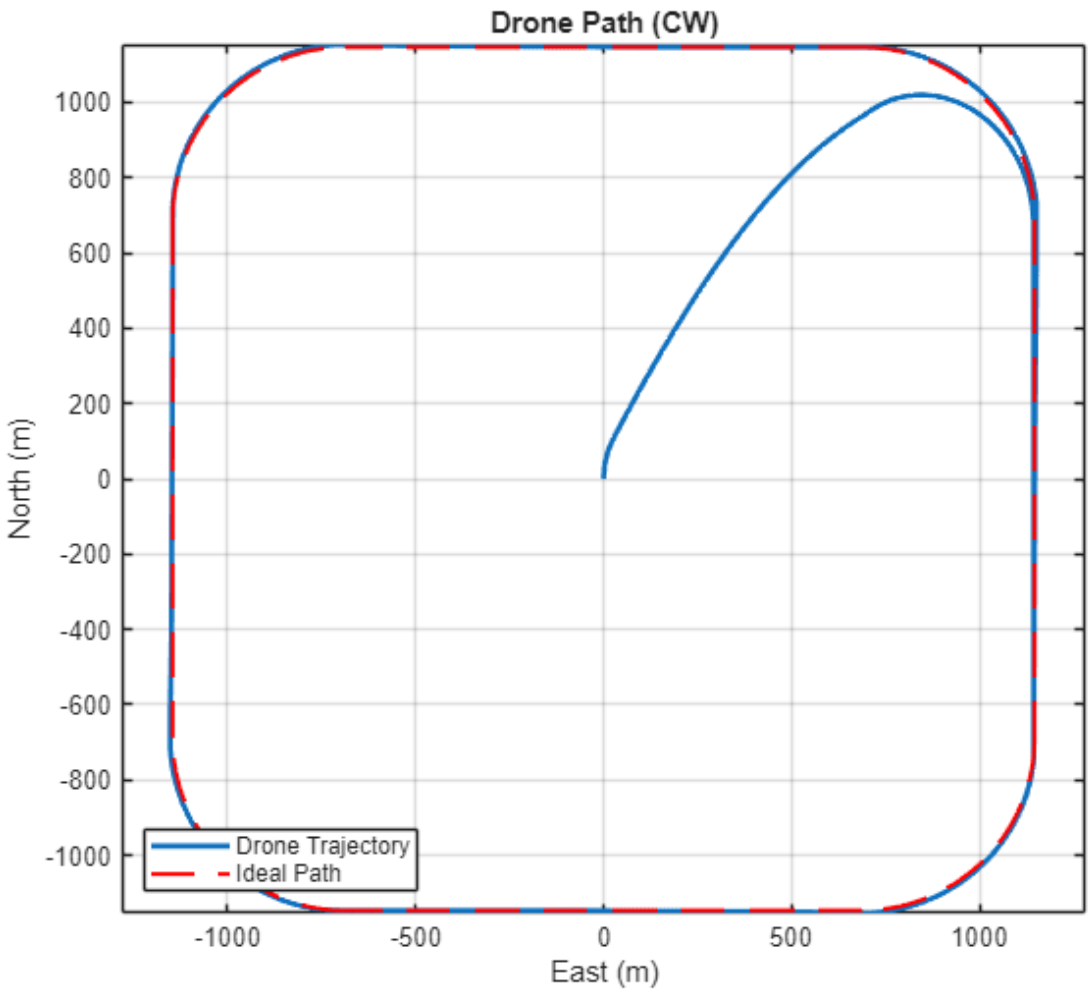
Vector Guidance

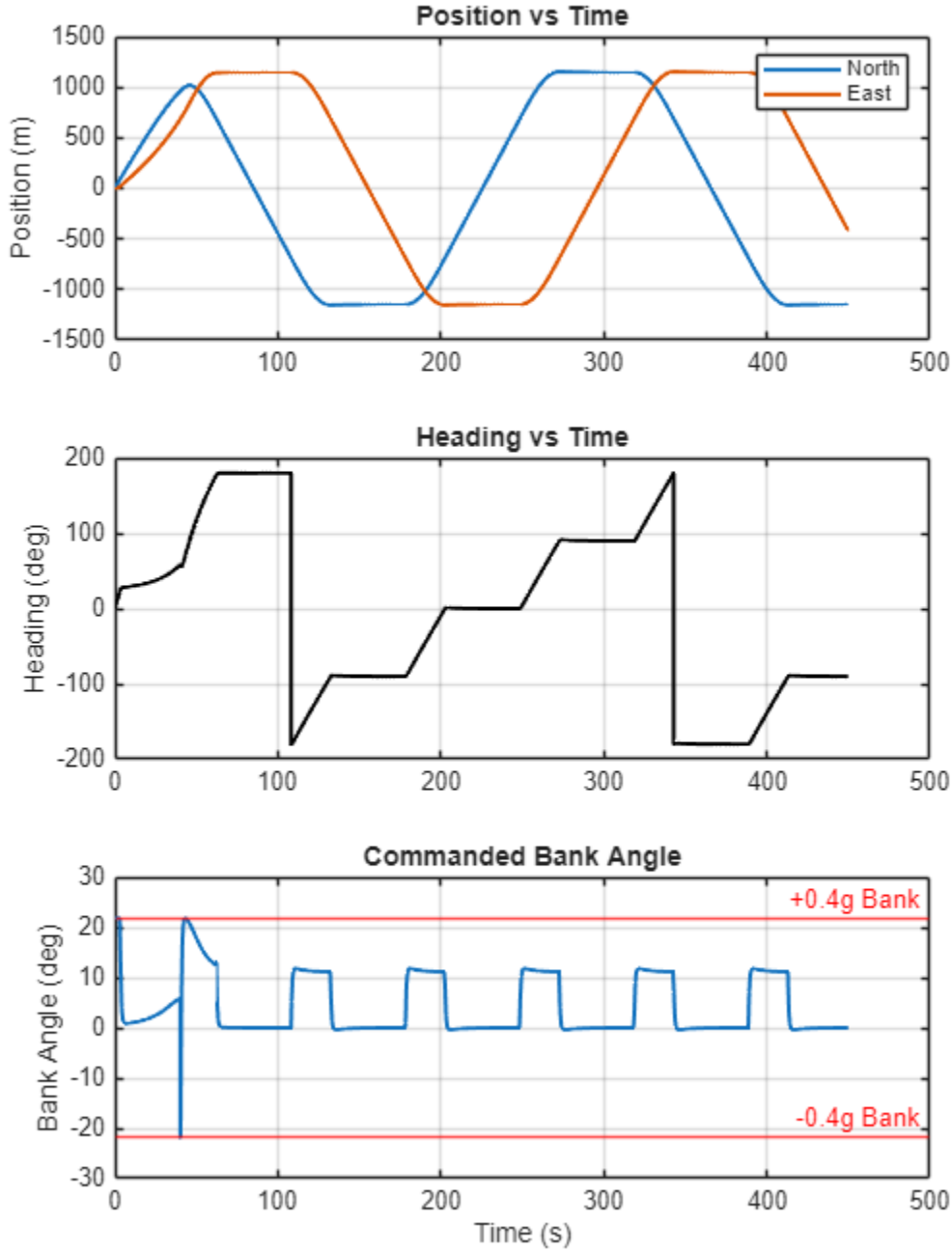
```
clear; close all; clc;
set(0, 'DefaultFigureWindowStyle', 'normal');

% params
g = 9.81;
V = 30;                     % Constant speed (m/s)
a_max = 0.4 * g;            % Max lateral acceleration
r_min = V^2 / a_max;        % Minimum turning radius (approx 229.36 m)
R = 2 * r_min;              % Corner radius
L = 10 * r_min;             % Square side length
half_L = L / 2;
phi_max = atan(0.4);        % Maximum allowed bank angle

% sim params
dt = 0.01;
time = 0:dt:450;            % Total simulation time
p = [0; 0];                 % Start at (0,0) North-East
chi = 0;                    % Start flying North (0 rad)
state = 1;                  % Initial State (1 = Line 1)

% data
path_history = zeros(length(time), 2);
chi_history = zeros(length(time), 1);
phi_history = zeros(length(time), 1);

for i = 1:length(time)

    % get commanded heading
    [chi_c, state] = getGuidance(p, state, L, R, r_min);

    % p controller
    diff = chi_c - chi;
    diff = mod(diff + pi, 2*pi) - pi; % wrap to [-pi, pi]

    chi_dot_cmd = diff * 2; % p gain here

    % get commanded bank angle
    ay_cmd = V * chi_dot_cmd;
    phi_c = atan(ay_cmd / g);

    % clamp
    phi_c = max(-phi_max, min(phi_max, phi_c));

    % actual turn rate
    chi_dot = (g / V) * tan(phi_c);

    %update
    chi = chi + chi_dot * dt;
    chi = mod(chi + pi, 2*pi) - pi;
    p(1) = p(1) + V * cos(chi) * dt; % North
```

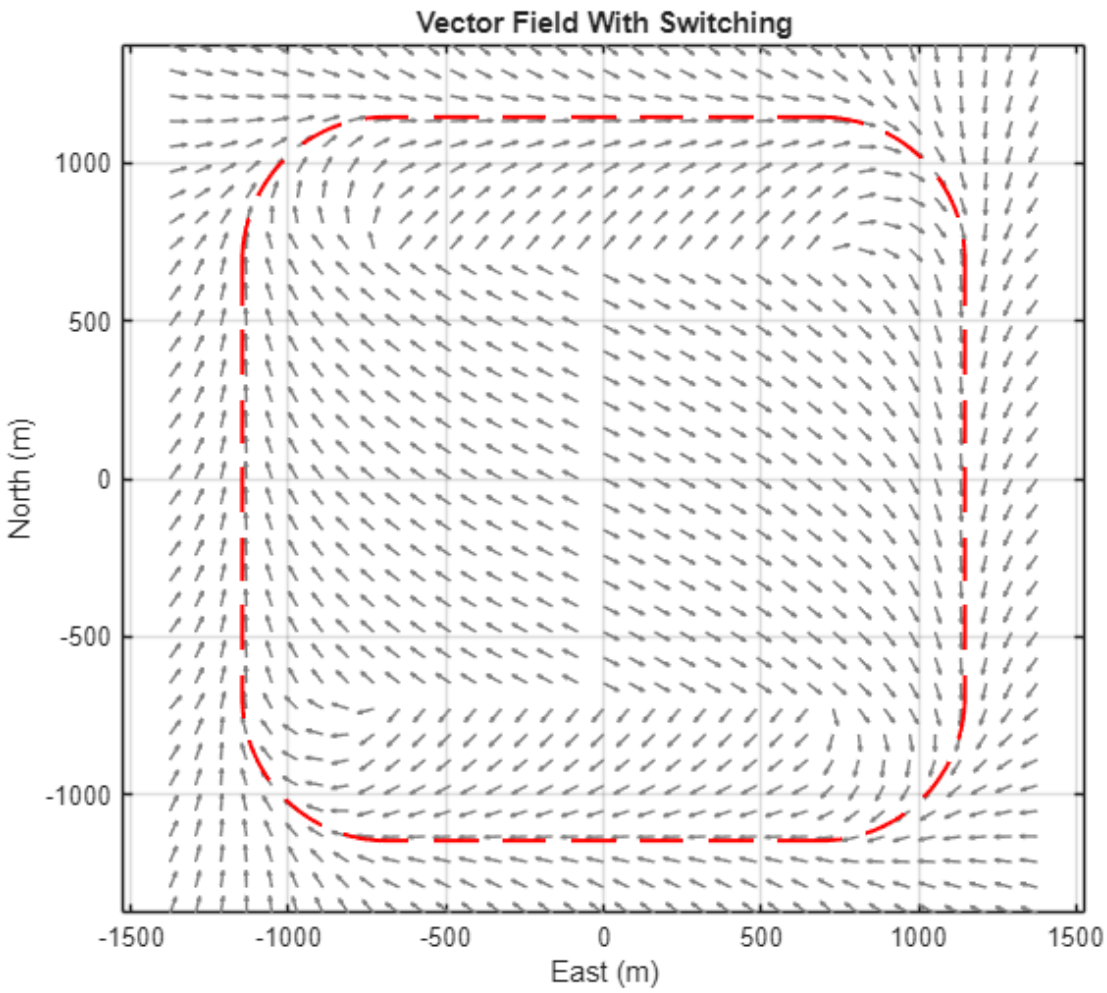
```
    p(2) = p(2) + V * sin(chi) * dt; % East

    % log
    path_history(i, :) = p';
    chi_history(i) = chi;
    phi_history(i) = phi_c;
end

% plotting

% path
figure('Name', 'Drone Trajectory', 'Position', [100, 100, 700, 600]);
plot(path_history(:,2), path_history(:,1), 'LineWidth', 2); hold on;
plotIdealPath(L, R); % Helper to plot the ideal rounded square
axis equal; grid on;
xlabel('East (m)'); ylabel('North (m)');
title('Drone Path (CW)');
legend('Drone Trajectory', 'Ideal Path', 'Location', 'southwest');

% time histories
figure('Name', 'Time Histories', 'Position', [850, 100, 600, 800]);
subplot(3,1,1);
plot(time, path_history(:,1), time, path_history(:,2), 'LineWidth', 1.5);
grid on; ylabel('Position (m)'); title('Position vs Time'); legend('North',
'East');

subplot(3,1,2);
plot(time, rad2deg(chi_history), 'k', 'LineWidth', 1.5);
grid on; ylabel('Heading (deg)'); title('Heading vs Time');

subplot(3,1,3);
plot(time, rad2deg(phi_history), 'LineWidth', 1.5); hold on;
yline(rad2deg(phi_max), 'r', '+0.4g Bank');
yline(-rad2deg(phi_max), 'r', '-0.4g Bank');
grid on; xlabel('Time (s)'); ylabel('Bank Angle (deg)'); title('Commanded
Bank Angle');

% vector field
figure('Name', 'Vector Field', 'Position', [1500, 100, 700, 600]);
[E_grid, N_grid] = meshgrid(linspace(-0.6*L, 0.6*L, 35), linspace(-0.6*L,
0.6*L, 35));
U = zeros(size(N_grid)); V_vf = zeros(size(N_grid));

for i = 1:numel(N_grid)
    pt = [N_grid(i); E_grid(i)];
    st = determineStateFromPos(pt, L, R); % Static region check
    xc = getGuidance(pt, st, L, R, r_min);
    U(i) = sin(xc); % East component
    V_vf(i) = cos(xc); % North component
end

quiver(E_grid, N_grid, U, V_vf, 0.5, 'color', [0.5 0.5 0.5]); hold on;
plotIdealPath(L, R);
axis equal; grid on;
```

```
xlabel('East (m)'); ylabel('North (m)');
title('Vector Field With Switching');

% functions

function [X_c, state] = getGuidance(p, current_state, L, R, r_min)
    % Manages switching between lines and orbits using Half-Planes
    half_L = L/2;
    state = current_state;

    % Corner Centers
    c1 = [half_L - R; half_L - R];   % Top-Right
    c2 = [-half_L + R; half_L - R];  % Bottom-Right
    c3 = [-half_L + R; -half_L + R]; % Bottom-Left
    c4 = [half_L - R; -half_L + R];  % Top-Left

    % State Machine
    if state == 1 && p(2) >= half_L - R && p(1) >= 0; state = 2; end
    if state == 2 && p(1) <= half_L - R && p(2) >= 0; state = 3; end
    if state == 3 && p(1) <= -half_L + R && p(2) >= 0; state = 4; end
    if state == 4 && p(2) <= half_L - R && p(1) <= 0; state = 5; end
    if state == 5 && p(2) <= -half_L + R && p(1) <= 0; state = 6; end
    if state == 6 && p(1) >= -half_L + R && p(2) <= 0; state = 7; end
    if state == 7 && p(1) >= half_L - R && p(2) <= 0; state = 8; end
    if state == 8 && p(2) >= -half_L + R && p(1) >= 0; state = 1; end

    % Vector Field Commands
    switch state
        case 1; X_c = followLine([half_L; 0], [0; 1], p, r_min);  % North
Edge, Fly East
        case 2; X_c = followOrbit(c1, R, 1, p);                   % TR Orbit
(CW = 1)
        case 3; X_c = followLine([0; half_L], [-1; 0], p, r_min); % East
Edge, Fly South
        case 4; X_c = followOrbit(c2, R, 1, p);                   % BR Orbit
(CW = 1)
        case 5; X_c = followLine([-half_L; 0], [0; -1], p, r_min);% South
Edge, Fly West
        case 6; X_c = followOrbit(c3, R, 1, p);                   % BL Orbit
(CW = 1)
        case 7; X_c = followLine([0; -half_L], [1; 0], p, r_min); % West
Edge, Fly North
        case 8; X_c = followOrbit(c4, R, 1, p);                   % TL Orbit
(CW = 1)
    end
end

function X_c = followLine(r, q, p, r_min)
    k_path = 1 / r_min;
    X_inf = 0.8 * (pi/2);
    X_q = atan2(q(2), q(1));
    e_py = -sin(X_q)*(p(1) - r(1)) + cos(X_q)*(p(2) - r(2));
    X_c = X_q - X_inf * (2/pi) * atan(k_path * e_py);
end
```

```
function X_c = followOrbit(c, radius, dir, p)
    k_orbit = 2.0; % this is smooth
    d = norm(p - c);
    psi = atan2(p(2) - c(2), p(1) - c(1));
    X_c = psi + dir * ( (pi/2) + atan(k_orbit * ((d - radius)/radius)) );
end

function st = determineStateFromPos(p, L, R)
    % Helper exclusively for plotting the vector field map
    half_L = L/2;
    if p(1) >= half_L - R
        if p(2) >= half_L - R; st = 2; elseif p(2) <= -half_L + R; st = 8;
else; st = 1; end
    elseif p(1) <= -half_L + R
        if p(2) >= half_L - R; st = 4; elseif p(2) <= -half_L + R; st = 6;
else; st = 5; end
    else
        if p(2) >= 0; st = 3; else; st = 7; end
    end
end

function plotIdealPath(L, R)
    half_L = L/2;
    centers = [half_L-R, half_L-R; -half_L+R, half_L-R; -half_L+R,
-half_L+R; half_L-R, -half_L+R];
    angs = [0, pi/2; pi/2, pi; pi, 3*pi/2; 3*pi/2, 2*pi];
    for i = 1:4
        th = linspace(angs(i,1), angs(i,2), 50);
        plot(centers(i,2) + R*sin(th), centers(i,1) + R*cos(th), 'r--',
'LineWidth', 1.5);
    end
    plot([half_L-R, -half_L+R], [half_L, half_L], 'r--', 'LineWidth', 1.5);
    plot([half_L-R, -half_L+R], [-half_L, -half_L], 'r--', 'LineWidth', 1.5);
    plot([half_L, half_L], [half_L-R, -half_L+R], 'r--', 'LineWidth', 1.5);
    plot([-half_L, -half_L], [half_L-R, -half_L+R], 'r--', 'LineWidth', 1.5);
end
```


Drone Path (CW)
1000
800
600
400
200
0
-200
-400
-600
-800
-1000
North (m)
-1000
-500
0
500
1000
East (m)
Drone Trajectory
Ideal Path


Position vs Time
Position (m)
North
East
Heading vs Time
Heading (deg)
Commanded Bank Angle
Bank Angle (deg)
+0.4g Bank
-0.4g Bank
Time (s)



SATELLITE STATE ESTIMATION

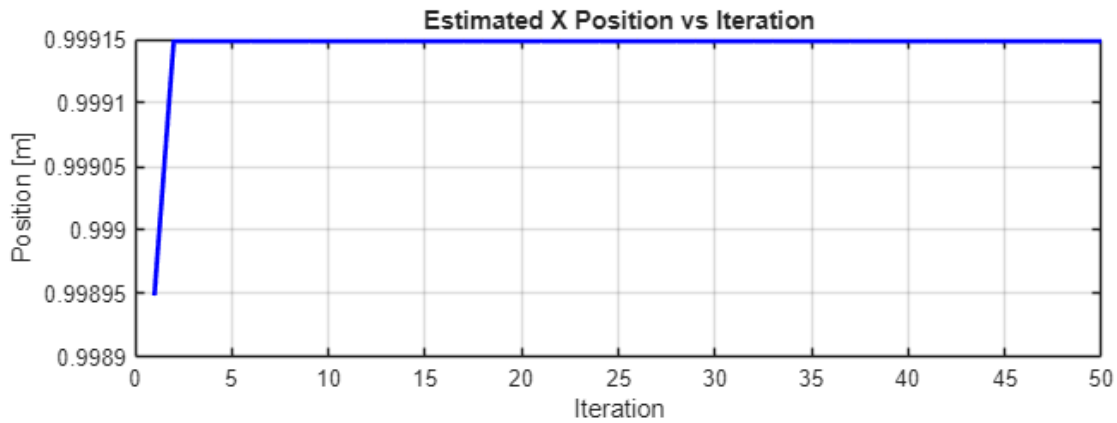
```
clear; close all; clc;

% 32 satellites: [id, pos_x, pos_y, vel_x, vel_y, range, range_rate]
data = [1,3746.4,9509.1,283.93,546.7,10218,607.9;
2,7320.9,5988.6,143.92,806.2,9456.2,616.84;
3,1561.2,1561.9,77.551,990.89,2206.2,750.46;
4,581.84,8663.8,775.24,202.72,8681.3,249.9;
5,6012.2,7082.7,8.5221,819.46,9288.2,625.29;
6,206.84,9701.1,709.86,733.01,9701.3,743.95;
7,8325.4,2125.4,774.27,78.045,8591.1,765.55;
8,1819.2,1836,361.47,119.87,2582.6,334.63;
9,3043.4,5249.6,866.1,627.3,6065.6,972.03;
10,4320.5,2914.3,333.9,67.558,5209.6,309.89;
11,6119.5,1396.9,313.98,329.18,6275.5,375.43;
12,2922.4,3665.6,732.61,641.56,4686,953.26;
13,4561.7,7853.8,890.21,476.21,9080.1,853.95;
```

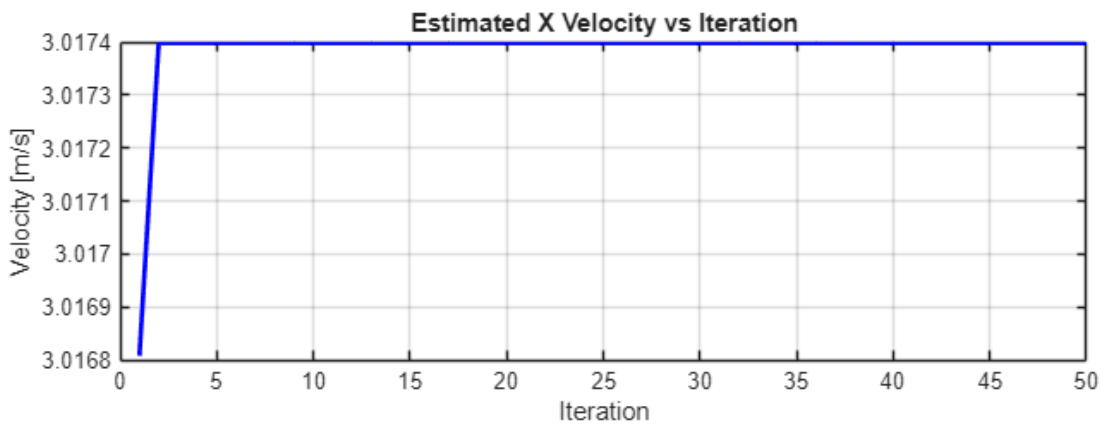
```
14,1997.7,5144.3,122.59,717.24,5516.6,708.08;
15,5925.1,466.5,763.79,565.28,5942.4,802.4;
16,6076.4,1707.2,773.97,497.8,6310.2,875.82;
17,651.52,9490.9,525.73,431.54,9511.1,462.38;
18,9657.3,8086,28.419,111.89,12593,88.764;
19,3047.1,978.72,34.429,640.41,3198.7,224.3;
20,6843.3,4403.5,317.36,512.57,8135.7,539.4;
21,1221.4,4953.8,910.57,253.29,5099.7,459.25;
22,344.89,9095.2,413.38,759.55,9099.7,770.33;
23,2588.8,6627.2,231.8,80.98,7112.7,154.88;
24,3118.1,5202.7,292.75,165.22,6063.3,287.07;
25,5468.1,1850.5,932.7,812.12,5771.2,1139.5;
26,9696.8,7753.3,636.4,875.46,12414,1038.8;
27,9396,8950.3,806.67,190.57,12974,710.54;
28,5980,9220.7,895.56,543.34,10988,938.33;
29,885.93,1961.8,810.44,900.09,2150.3,1149;
30,453.27,3255.3,321,114.05,3284.8,152.81;
31,3887.8,2715.5,230.94,431.11,4740.3,431.19;
32,8288.4,3569.5,821.01,864.73,9022.4,1091.8];

% Run the estimation
[x_final, x_hist, P_post] = run_estimation(data);

% Plotting

% x estimates
iterations = 1:50;
figure('Name', 'X State Estimates', 'Position', [100 100 800 600]);

subplot(2,1,1);
plot(iterations, x_hist(1,:), 'b-', 'LineWidth', 2); hold on;
title('Estimated X Position vs Iteration');
xlabel('Iteration'); ylabel('Position [m]');
grid on;

subplot(2,1,2);
plot(iterations, x_hist(3,:), 'b-', 'LineWidth', 2); hold on;
title('Estimated X Velocity vs Iteration');
xlabel('Iteration'); ylabel('Velocity [m/s]');
grid on;


% y estimates
figure('Name', 'Y State Estimates', 'Position', [100 100 800 600]);
subplot(2,1,1);
plot(iterations, x_hist(2,:), 'r-', 'LineWidth', 2);
title('Estimated Y Position vs Iteration');


subplot(2,1,2);
plot(iterations, x_hist(4,:), 'r-', 'LineWidth', 2);
title('Estimated Y Velocity vs Iteration');
```

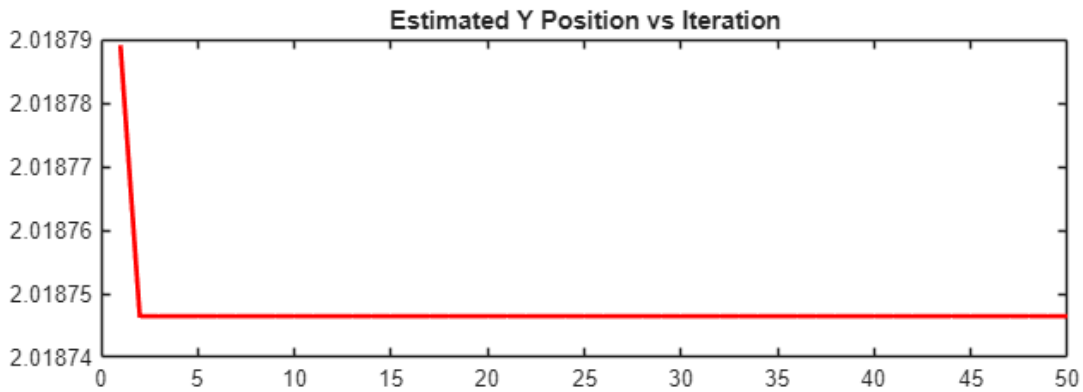
```
%Plot Posterior Density of Position
mu_pos = x_final(1:2);
Sigma_pos = P_post(1:2, 1:2);
plot_density(mu_pos, Sigma_pos, ...
    'Conditional Posterior Density of Position', 'X Position [m]', 'Y
Position [m]');

%Plot Posterior Density of Velocity
mu_vel = x_final(3:4);
Sigma_vel = P_post(3:4, 3:4);
plot_density(mu_vel, Sigma_vel, ...
    'Conditional Posterior Density of Velocity', 'X Velocity [m/s]', 'Y
Velocity [m/s]');


%Functions

function [x_est, x_hist, P_post] = run_estimation(data)
    % Initial conditions and weights
    x_est = [0; 0; 0; 0];
    %x_prior = [0; 0; 0; 0];

    range_noise_std = 0.5;
    estimate_std = 1.0;

    %W_prior = (1 / estimate_std^2) * eye(4);
    W_meas  = (1 / range_noise_std^2);

    x_hist = zeros(4, 50);

    for iter = 1:50

        % Keep if Satellite X >= Receiver Estimated X
        trust_idx = find(data(:, 2) >= x_est(1));
        n_active = length(trust_idx);

        H = zeros(2*n_active, 4);
        delta_z = zeros(2*n_active, 1);

        for k = 1:n_active
            idx = trust_idx(k);
            pos = data(idx, 2:3)';        % [x_i; y_i]
            vel = data(idx, 4:5)';        % [x_dot_i; y_dot_i]
            meas_range = data(idx, 6);    % rho_i
            meas_rate = data(idx, 7);     % rho_dot_i

            % Compute relative vectors
            d_x = pos - x_est(1:2);
            d_v = vel - x_est(3:4);

            r = norm(d_x);
            u = d_x / r; %Unit vector pointing from receiver to satellite
```

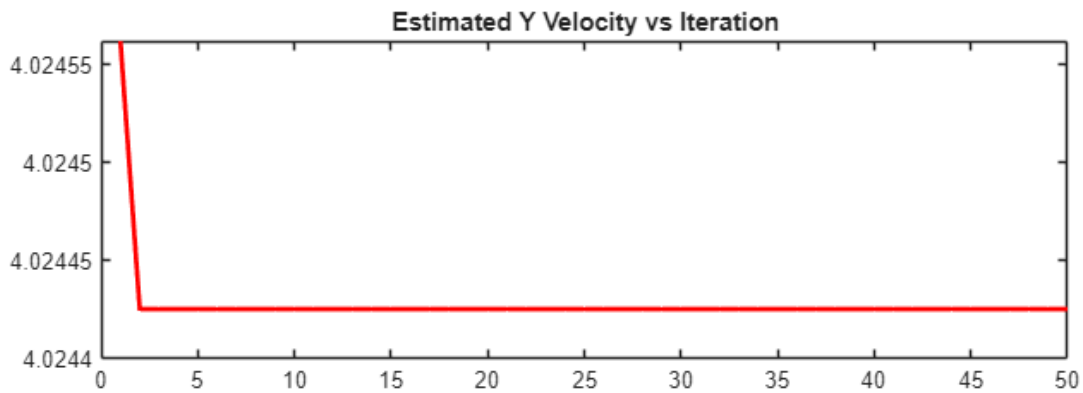
```
            % Predicted measurements
            z_range_i = r;
            z_rate_i = dot(d_v, u);

            % Residuals
            delta_z(2*k - 1) = meas_range - z_range_i;
            delta_z(2*k)     = meas_rate - z_rate_i;

            H(2*k - 1, :) = [-u(1), -u(2), 0, 0];

            % H_rate: Partial of range rate wrt state
            H(2*k, :) = [ -(d_v(1) - z_rate_i * u(1))/r, ...
                          -(d_v(2) - z_rate_i * u(2))/r, ...
                          -u(1), -u(2) ];
        end

        % update step
        W_matrix = W_meas * eye(2*n_active);

        %info_matrix = H' * W_matrix * H + W_prior;
        info_matrix = H' * W_matrix * H;
        gradient = H' * W_matrix * delta_z;
        %gradient = H' * W_matrix * delta_z - W_prior * (x_est - x_prior);

        % Solve for state delta
        delta_x = info_matrix \ gradient;
        %disp(norm(delta_x))

        % Update estimate
        x_est = x_est + delta_x;

        % Store history
        x_hist(:, iter) = x_est;
    end

    P_post = inv(info_matrix);
end

function plot_density(mu, Sigma, title_str, xlabel_str, ylabel_str)
    sig_x = sqrt(Sigma(1,1));
    sig_y = sqrt(Sigma(2,2));

    x_vals = linspace(mu(1) - 3*sig_x, mu(1) + 3*sig_x, 100);
    y_vals = linspace(mu(2) - 3*sig_y, mu(2) + 3*sig_y, 100);
    [X, Y] = meshgrid(x_vals, y_vals);

    Z = zeros(size(X));
    inv_Sigma = inv(Sigma);
    det_Sigma = det(Sigma);
    norm_const = 1 / (2 * pi * sqrt(det_Sigma));

    for i = 1:size(X,1)
        for j = 1:size(X,2)
```

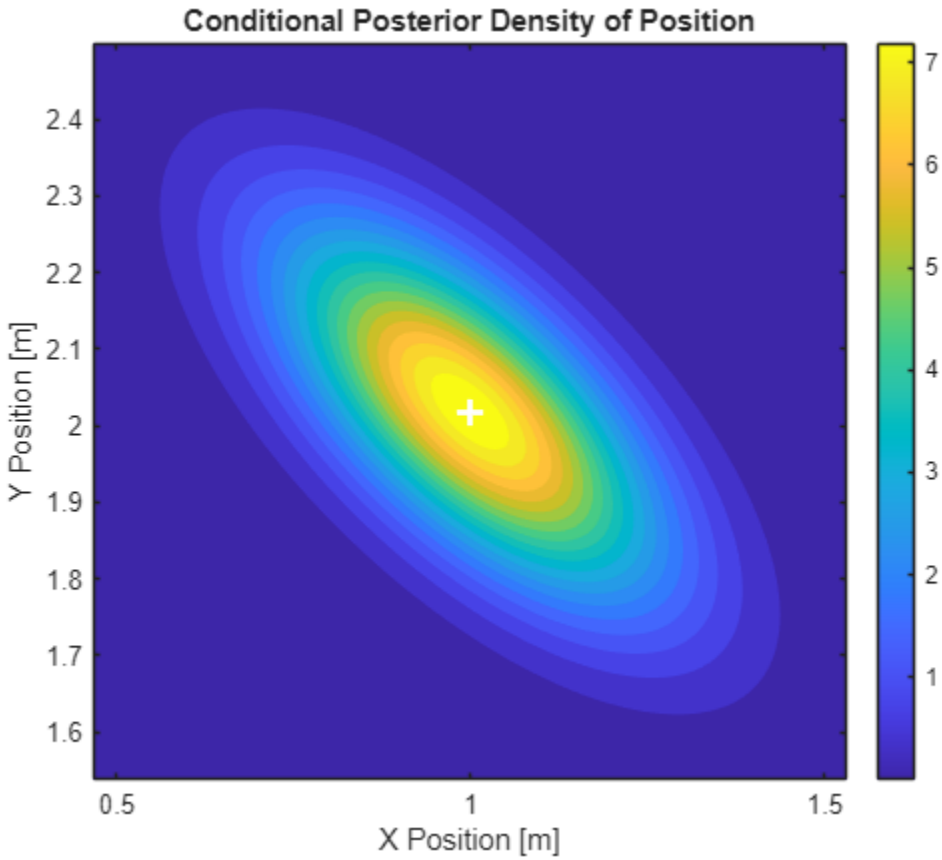
```
            diff = [X(i,j) - mu(1); Y(i,j) - mu(2)];
            Z(i,j) = norm_const * exp(-0.5 * diff' * inv_Sigma * diff);
        end
    end

    % Create the contour plot
    figure('Name', title_str, 'Position', [100 100 600 500]);
    contourf(X, Y, Z, 20, 'LineStyle', 'none');
    colorbar;
    hold on;
    % Mark the mean (Final Estimate)
    plot(mu(1), mu(2), 'w+', 'MarkerSize', 10, 'LineWidth', 2);

    title(title_str);
    xlabel(xlabel_str);
    ylabel(ylabel_str);
end
```

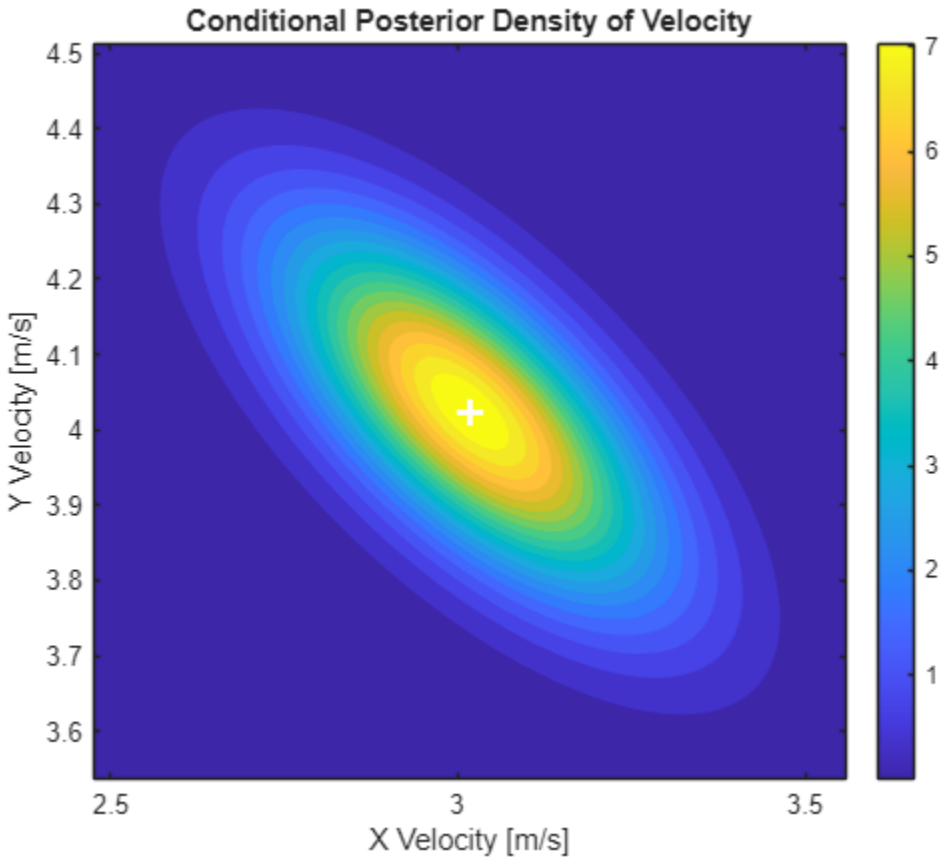





Estimated Y Position vs Iteration
2.01879
2.01878
2.01877
2.01876
2.01875
2.01874
0
5
10
15
20
25
30
35
40
45
50


Estimated Y Velocity vs Iteration
4.02455
4.0245
4.02445
4.0244
0
5
10
15
20
25
30
35
40
45
50


Conditional Posterior Density of Position
Y Position [m]
2.4
2.3
2.2
2.1
2
1.9
1.8
1.7
1.6
0.5
1
1.5
X Position [m]
7
6
5
4
3
2
1



Published with MATLAB® R2025b